This document summarizes the equations used in the lidar-image teach and repeat navigation project. The main references of this document are Anderson and Barfoot (2013); Anderson et al. (2015); Barfoot (2024); Tong et al. (2014).

1 Lidar Image Visual Odometry

In this section, we summarize the necessary equations for the lidar image-based visual odometry. In particular, we focus on lidar intensity images and range images.

1.1 Motion Compensated RANSAC

We define the inertial frame and lidar sensor frame as $\mathcal{F}_i$ and $\mathcal{F}_s$. The lidar sensor frame at timestamp t_j is denoted as $\mathcal{F}_{st_j}$. We consider two sets of corresponding measurements, $\mathbf{y}_{m,1}$ and $\mathbf{y}_{m,2}$ with $m = 1, \cdots, M$, extracted from sequential images at time $t_{m,1}$ and $t_{m,2}$. For landmark $\mathbf{p}_m$ in the inertial frame, we have

$$\begin{aligned} \mathbf{p}_{m1} &= \mathbf{T}_{si}(t_{m,1})\mathbf{p}_m \\ \mathbf{p}_{m2} &= \mathbf{T}_{si}(t_{m,2})\mathbf{p}_m. \end{aligned} \tag{1}$$

The lidar image sensor model is

$$\begin{aligned} \mathbf{y}_{m1} &= f\left(\mathbf{T}_{si}(t_{m,1})\mathbf{p}_m\right) + \mathbf{n}_1 \\ \mathbf{y}_{m2} &= f\left(\mathbf{T}_{si}(t_{m,2})\mathbf{p}_m\right) + \mathbf{n}_2, \end{aligned} \tag{2}$$

with $\mathbf{n}_1 \sim \mathcal{N}(\mathbf{0}, \mathbf{R}_{m,1})$ and $\mathbf{n}_2 \sim \mathcal{N}(\mathbf{0}, \mathbf{R}_{m,2})$.

The incremental motion is

$$\mathbf{T}^m_{st_2,st_1} = \mathbf{T}_{si}(t_{m,2})\mathbf{T}^{-1}_{si}(t_{m,1}). \tag{3}$$

The estimated position of landmark m in $\mathcal{F}_{st_2}$ is

$$\hat{\mathbf{p}}_{m,2} = \mathbf{T}^m_{st_2,st_1}\mathbf{p}_{m,1}. \tag{4}$$

For the motion-compensated RANSAC, we put a constant velocity motion prior $\boldsymbol{\varpi} = \begin{bmatrix} \mathbf{v} \\ \mathbf{w} \end{bmatrix}$ and the motion model is

$$\dot{\mathbf{T}}_{si}(t) = \boldsymbol{\varpi}^{\wedge}\mathbf{T}_{si}(t). \tag{5}$$

If we ignore the noise $\mathbf{n}_1$ in sensor frame 1, we have

$$\mathbf{y}_{m,1} = f\left(\mathbf{T}_{si}(t_{m,1})\mathbf{p}_m\right) = f\left(\mathbf{p}_{m,1}\right). \tag{6}$$

Then we have

$$\mathbf{p}_{m,1} = f^{-1}\left(\mathbf{y}_{m,1}\right) \tag{7}$$

last revised: October 27, 2026

and the estimated position of landmark m in $\mathcal{F}_{st_2}$ is

$$\hat{\mathbf{p}}_{m,2} = \mathbf{T}^{m}_{st_2,st_1} f^{-1}\left(\mathbf{y}_{m,1}\right). \tag{8}$$

The relative motion is

$$\begin{aligned} \mathbf{T}_{si}(t_2) &= \mathbf{T}_{st_2,st_1}\mathbf{T}_{si}(t_1), \\ \boldsymbol{\xi} &= \boldsymbol{\varpi}(t_2 - t_1) = \boldsymbol{\varpi}\Delta t, \end{aligned} \tag{9}$$

and

$$\mathbf{T}_{st_2,st_1} = \exp((\boldsymbol{\varpi}\Delta t)^{\wedge}) = \exp(\xi^{\wedge}). \tag{10}$$

For landmark m, we have the error

$$\begin{aligned} \mathbf{e}_m &= \mathbf{y}_{m,2} - \hat{\mathbf{y}}_{m,2} \\ &= \mathbf{y}_{m,2} - f\left(\mathbf{T}^{m}_{st_2,st_1} f^{-1}(\mathbf{y}_{m,1})\right) \\ &= \mathbf{y}_{m,2} - f\left(\mathbf{T}^{m}_{st_2,st_1}\mathbf{p}_{m,1}\right) \\ &= \mathbf{y}_{m,2} - f\left(\exp\left((\boldsymbol{\varpi}\Delta t)^{\wedge}\right)\mathbf{p}_{m,1}\right) \\ &= \mathbf{y}_{m,2} - f(g(\boldsymbol{\varpi})) \end{aligned} \tag{11}$$

$$\begin{aligned} \mathbf{T}_{st_2,st_1} &= \exp\left((\bar{\boldsymbol{\xi}} + \delta\boldsymbol{\xi})^{\wedge}\right) \\ &= \exp\left(((\bar{\boldsymbol{\varpi}} + \delta\boldsymbol{\varpi})\Delta t)^{\wedge}\right) \\ &\approx \exp\left((\mathcal{J}_l(\bar{\xi})\Delta t\Delta\boldsymbol{\varpi})^{\wedge}\right)\exp\left(\bar{\xi}^{\wedge}\right) \\ &= \exp\left((\mathcal{J}_l(\bar{\xi})\Delta t\Delta\boldsymbol{\varpi})^{\wedge}\right)\bar{\mathbf{T}}_{st_2,st_1}. \end{aligned} \tag{12}$$

We treat the $\bar{\mathbf{p}}_{m,1}$ computation to be noise-less

$$\begin{aligned} g(\boldsymbol{\varpi}) &= \mathbf{T}^{m}_{st_2,st_1}\bar{\mathbf{p}}_{m,1} \\ &= \exp\left((\mathcal{J}_l(\bar{\xi})\Delta t\Delta\boldsymbol{\varpi})^{\wedge}\right)\bar{\mathbf{T}}_{st_2,st_1}\bar{\mathbf{p}}_{m,1} \end{aligned} \tag{13}$$

where $\mathcal{J}_l(\cdot)$ is the left Jacobian Barfoot (2024). Define $\boldsymbol{\eta} = \mathcal{J}_l(\bar{\xi})\Delta t\Delta\boldsymbol{\varpi}$ and $\bar{\mathbf{g}} = \bar{\mathbf{T}}_{st_2,st_1}\bar{\mathbf{p}}_{m,1}$, we have

$$\begin{aligned} g(\boldsymbol{\varpi}) &= \exp\left(\boldsymbol{\eta}^{\wedge}\right)\bar{\mathbf{g}} \\ &\approx \left(1 + \boldsymbol{\eta}^{\wedge}\right)\bar{\mathbf{g}} \\ &= \bar{\mathbf{g}} + \boldsymbol{\eta}^{\wedge}\bar{\mathbf{g}} \\ &= \bar{\mathbf{g}} + \bar{\mathbf{g}}^{\odot}\boldsymbol{\eta} \\ &= \bar{\mathbf{g}} + \bar{\mathbf{g}}^{\odot}\mathcal{J}_l(\bar{\xi})\Delta t\Delta\boldsymbol{\varpi}. \end{aligned} \tag{14}$$

Hence, we have

$$\mathbf{e}_m \approx \bar{\mathbf{e}}_m - \mathbf{H}\delta\boldsymbol{\varpi} \tag{15}$$

where the jacobian $\mathbf{H}_m$ is

$$\begin{aligned}\mathbf{H}_m &= \left.\frac{\partial f}{\partial g}\right|_{\bar{\mathbf{g}}_m} \bar{\mathbf{g}}^{\odot}\mathcal{J}_l(\bar{\xi})\Delta t \\ &= \mathbf{F}_m\bar{\mathbf{g}}^{\odot}\mathcal{J}_l(\bar{\xi})\Delta t.\end{aligned} \tag{16}$$

Then we can solve for the $\boldsymbol{\varpi}$ incrementally through Gauss-Newton solver with

$$\left(\sum_m \mathbf{H}_m^T\mathbf{R}_{m,2}^{-1}\mathbf{H}_m\right)\delta\boldsymbol{\varpi} = \sum_m \mathbf{H}_m^T\mathbf{R}_{m,2}^{-1}\bar{\mathbf{e}}_m. \tag{17}$$

2 Degeneracy-Aware Complementary Fusion

In this section, we recap the degeneracy detection for LiDAR geometric localization and derive how a complementary sensing modality, the LiDAR intensity image in our case, can be introduced only in the geometrically degenerate directions.

2.1 Recap of LiDAR Localization Degeneracy Detection

We formulate the weighted least-squares problem using the LiDAR geometric features as

$$\begin{aligned}&\min_{\delta\mathbf{x}} \|\mathbf{r}_g - \mathbf{J}_g\delta\mathbf{x}\|^2_{\mathbf{W}_g} \\ =&\min_{\delta\mathbf{x}} \left(\mathbf{r}_g - \mathbf{J}_g\delta\mathbf{x}\right)^\top \mathbf{W}_g \left(\mathbf{r}_g - \mathbf{J}_g\delta\mathbf{x}\right) \\ =&\min_{\delta\mathbf{x}} \underbrace{\mathbf{r}_g^\top\mathbf{W}_g\mathbf{r}_g}_{\text{const. w.r.t. }\delta\mathbf{x}} - 2\mathbf{r}_g^\top\mathbf{W}_g\mathbf{J}_g\delta\mathbf{x} + \delta\mathbf{x}^\top\mathbf{J}_g^\top\mathbf{W}_g\mathbf{J}_g\delta\mathbf{x} \\ =&\min_{\delta\mathbf{x}} \delta\mathbf{x}^\top\mathbf{J}_g^\top\mathbf{W}_g\mathbf{J}_g\delta\mathbf{x} - 2\mathbf{r}_g^\top\mathbf{W}_g\mathbf{J}_g\delta\mathbf{x}.\end{aligned} \tag{18}$$

Define the Hessian matrix and the gradient vector as

$$\mathbf{H}_g = \mathbf{J}_g^\top\mathbf{W}_g\mathbf{J}_g, \qquad \mathbf{b}_g^\top = \mathbf{r}_g^\top\mathbf{W}_g\mathbf{J}_g \;\rightarrow\; \mathbf{b}_g = \mathbf{J}_g^\top\mathbf{W}_g\mathbf{r}_g. \tag{19}$$

We now have

$$\min_{\delta\mathbf{x}} \delta\mathbf{x}^\top\mathbf{H}_g\delta\mathbf{x} - 2\mathbf{b}_g^\top\delta\mathbf{x} \quad\Leftrightarrow\quad \min_{\delta\mathbf{x}} \frac{1}{2}\delta\mathbf{x}^\top\mathbf{H}_g\delta\mathbf{x} - \mathbf{b}_g^\top\delta\mathbf{x}. \tag{20}$$

Setting the gradient w.r.t. $\delta\mathbf{x}$ to zero, we have the one-step update

$$\mathbf{H}_g\delta\mathbf{x} = \mathbf{b}_g. \tag{21}$$

To detect degeneracy, we perform the eigendecomposition of $\mathbf{H}_g$,

$$\mathbf{H}_g = \mathbf{V}\mathbf{\Lambda}_g\mathbf{V}^\top, \qquad \mathbf{\Lambda}_g = \operatorname{diag}\left(\lambda_1^g, \ldots, \lambda_6^g\right), \tag{22}$$

with the eigenvalues sorted in descending order. We treat the eigenvalue λ_i^g (and its eigenvector $\mathbf{v}_i$) as degenerate if

$$\frac{\lambda_{\max}^g}{\lambda_i^g} \geq \gamma, \tag{23}$$

where γ is a user-defined condition threshold. Assuming the first m directions are well-conditioned, we partition the eigenvectors into

$$\mathbf{V}_c = \left[\mathbf{v}_1, \ldots, \mathbf{v}_m\right], \qquad \mathbf{V}_d = \left[\mathbf{v}_{m+1}, \ldots, \mathbf{v}_6\right]. \tag{24}$$

The degenerate space is

$$\mathcal{D} = \operatorname{span}\left(\mathbf{V}_d\right). \tag{25}$$

Since the columns of $\mathbf{V}_d$ are orthonormal eigenvectors, $\mathbf{V}_d^\top\mathbf{V}_d = \mathbf{I}$. We can project the perturbation into $\mathcal{D}$ with

$$\delta\mathbf{x}_d = \mathbf{V}_d^\top\delta\mathbf{x}. \tag{26}$$

Conversely, for a perturbation lying in $\mathcal{D}$, $\mathbf{V}_d^\top\mathbf{V}_d\delta\mathbf{x}_d = \mathbf{V}_d^\top\delta\mathbf{x}$, and we have the inverse relationship

$$\delta\mathbf{x} = \mathbf{V}_d\,\delta\mathbf{x}_d. \tag{27}$$

2.2 Complementary Information in the Degenerate Space

We now bring the projection back to the original optimization formulation. If we have a complementary sensing modality, the LiDAR intensity image in our case, we have

$$\min_{\delta\mathbf{x}} \frac{1}{2}\delta\mathbf{x}^\top\mathbf{H}_v\delta\mathbf{x} - \mathbf{b}_v^\top\delta\mathbf{x}. \tag{28}$$

Substituting $\delta\mathbf{x} = \mathbf{V}_d\,\delta\mathbf{x}_d$ gives

$$\min_{\delta\mathbf{x}_d} \frac{1}{2}\delta\mathbf{x}_d^\top\underbrace{\mathbf{V}_d^\top\mathbf{H}_v\mathbf{V}_d}_{\mathbf{H}_{v,d}}\delta\mathbf{x}_d - \underbrace{\mathbf{b}_v^\top\mathbf{V}_d}_{\mathbf{b}_{v,d}^\top}\delta\mathbf{x}_d. \tag{29}$$

The projected Hessian $\mathbf{H}_{v,d}$ quantifies **how much information the complementary sensor contributes to the geometric degenerate space** $\mathcal{D}$ of the main sensor. Setting the gradient w.r.t. $\delta\mathbf{x}_d$ to zero, we have

$$\mathbf{H}_{v,d}\delta\mathbf{x}_d = \mathbf{b}_{v,d}, \qquad \text{with} \quad \begin{cases} \mathbf{H}_{v,d} = \mathbf{V}_d^\top\mathbf{H}_v\mathbf{V}_d, \\ \mathbf{b}_{v,d} = \mathbf{V}_d^\top\mathbf{b}_v. \end{cases} \tag{30}$$

2.3 Joint Optimization Restricted to the Degenerate Space

If we consider a joint optimization with both the LiDAR geometry and intensity features, we have

$$\min_{\delta\mathbf{x}} \|\mathbf{r}_g - \mathbf{J}_g\delta\mathbf{x}\|^2_{\mathbf{W}_g} + \|\mathbf{r}_v - \mathbf{J}_v\delta\mathbf{x}\|^2_{\mathbf{W}_v}. \tag{31}$$

Setting the gradient to zero, we have

$$\mathbf{H}_g\delta\mathbf{x} + \mathbf{H}_v\delta\mathbf{x} = \mathbf{b}_g + \mathbf{b}_v. \tag{32}$$

If we want the intensity image to contribute only to the geometric degenerate space $\mathcal{D}$, we can instead solve

$$\left(\mathbf{H}_g + \alpha\,\mathbf{V}_d\mathbf{V}_d^\top\mathbf{H}_v\mathbf{V}_d\mathbf{V}_d^\top\right)\delta\mathbf{x} = \mathbf{b}_g + \alpha\,\mathbf{V}_d\mathbf{V}_d^\top\mathbf{b}_v. \tag{33}$$

With the projection matrix $\mathbf{P}_d = \mathbf{V}_d\mathbf{V}_d^\top$, this is

$$\left(\mathbf{H}_g + \alpha\,\mathbf{P}_d\mathbf{H}_v\mathbf{P}_d\right)\delta\mathbf{x} = \mathbf{b}_g + \alpha\,\mathbf{P}_d\mathbf{b}_v, \tag{34}$$

where α is a parameter that balances the weights of the LiDAR geometry residuals and the LiDAR intensity feature residuals.

Derivation:

$$\min_{\delta\mathbf{x}_d} \frac{1}{2}\delta\mathbf{x}_d^\top\mathbf{V}_d^\top\mathbf{H}_v\mathbf{V}_d\,\delta\mathbf{x}_d - \mathbf{b}_v^\top\mathbf{V}_d\,\delta\mathbf{x}_d, \tag{35}$$

and substituting $\delta\mathbf{x}_d = \mathbf{V}_d^\top\delta\mathbf{x}$, we obtain the cost expressed in the full state perturbation

$$\min_{\delta\mathbf{x}} \frac{1}{2}\delta\mathbf{x}^\top \underbrace{\mathbf{V}_d\mathbf{V}_d^\top\mathbf{H}_v\mathbf{V}_d\mathbf{V}_d^\top}_{\mathbf{P}_d\mathbf{H}_v\mathbf{P}_d}\delta\mathbf{x} - \underbrace{\mathbf{b}_v^\top\mathbf{V}_d\mathbf{V}_d^\top}_{(\mathbf{P}_d\mathbf{b}_v)^\top}\delta\mathbf{x}. \tag{36}$$

Weighting this cost by α, adding it to the geometric cost $\frac{1}{2}\delta\mathbf{x}^\top\mathbf{H}_g\delta\mathbf{x} - \mathbf{b}_g^\top\delta\mathbf{x}$, and setting the gradient to zero (using $\mathbf{P}_d^\top = \mathbf{P}_d$) yields the normal equation above.

2.4 Intensity-Image Feature Residual Formulation

We first formulate the residual directly in the 2D intensity-image space and derive its covariance by propagating the measurement uncertainty of both matched features. We denote the LiDAR spherical projection model as $f(\cdot)$. A landmark $\mathbf{p}$ expressed in the sensor frame projects onto the intensity image as

$$\mathbf{y} = f\left(\mathbf{p}\right) = \begin{bmatrix} u \\ v \end{bmatrix}, \tag{37}$$

where (u, v) are the image coordinates, which correspond to the azimuth and elevation angles of the LiDAR beam. Unlike a camera, the LiDAR also provides the range ρ of every intensity-image feature. Throughout this section, we use i to index a feature in the first frame (time t_1) and j to index its match in the second

frame (time t_2).

2.4.1 3D point covariance of an intensity-image feature

Let $\mathbf{\Sigma}_{uv,i}$ be the image-space noise covariance of feature i and $\sigma^2_{\rho,i}$ its range noise variance. Stacking the image coordinates and the range gives the full measurement

$$\mathbf{z}_i = \begin{bmatrix} u_i \\ v_i \\ \rho_i \end{bmatrix}, \qquad \mathbf{\Sigma}_{\mathbf{z}_i} = \begin{bmatrix} \mathbf{\Sigma}_{uv,i} & \mathbf{0} \\ \mathbf{0}^\top & \sigma^2_{\rho,i} \end{bmatrix}. \tag{38}$$

The 3D point is recovered by the inverse projection, whose first-order perturbation is

$$\mathbf{p}_i = f^{-1}(\mathbf{z}_i), \qquad \delta\mathbf{p}_i \approx \mathbf{J}_{f^{-1},i}\,\delta\mathbf{z}_i, \qquad \mathbf{J}_{f^{-1},i} = \frac{\partial f^{-1}}{\partial \mathbf{z}_i} \in \mathbb{R}^{3\times 3}. \tag{39}$$

The covariance of the back-projected 3D point is therefore

$$\mathbf{\Sigma}_{\mathbf{p}_i} = \mathbf{J}_{f^{-1},i}\,\mathbf{\Sigma}_{\mathbf{z}_i}\,\mathbf{J}_{f^{-1},i}{}^\top. \tag{40}$$

The same expression holds for any feature j in the second frame by replacing the index i with j.

Note: the learnt uncertainty $\mathbf{\Sigma}_{uv}$ is essentially the uncertainty of the **azimuth and elevation angles**. After back-projection, it is scaled by the range, so $\mathbf{\Sigma}_{\mathbf{p}_i}$ grows perpendicular to the beam for distant points, while the uncertainty along the beam is governed by $\sigma^2_{\rho,i}$.

2.4.2 Predicted measurement in the second frame

Given the relative pose $\{\mathbf{R}_{t_2,t_1}, \mathbf{t}^{t_1,t_2}_{t_2}\}$, feature i is transformed into the second frame as

$$\hat{\mathbf{p}}_j = \mathbf{R}_{t_2,t_1}\mathbf{p}_i + \mathbf{t}^{t_1,t_2}_{t_2}, \qquad \mathbf{\Sigma}_{\hat{\mathbf{p}}_j} = \mathbf{R}_{t_2,t_1}\,\mathbf{\Sigma}_{\mathbf{p}_i}\mathbf{R}^\top_{t_2,t_1} = \mathbf{R}_{t_2,t_1}\mathbf{J}_{f^{-1},i}\,\mathbf{\Sigma}_{\mathbf{z}_i}\,\mathbf{J}_{f^{-1},i}{}^\top\mathbf{R}^\top_{t_2,t_1}. \tag{41}$$

The 2D feature residual compares the observed image coordinates of the match j with the reprojection of $\hat{\mathbf{p}}_j$,

$$\mathbf{r}_{ij} = \mathbf{y}_j - f(\hat{\mathbf{p}}_j) = \mathbf{y}_j - \hat{\mathbf{y}}_j. \tag{42}$$

Linearizing the projection,

$$\delta\hat{\mathbf{y}}_j = \mathbf{J}_{f,j}\,\delta\hat{\mathbf{p}}_j, \qquad \mathbf{J}_{f,j} = \frac{\partial f(\hat{\mathbf{p}}_j)}{\partial \hat{\mathbf{p}}_j} \in \mathbb{R}^{2\times 3}, \tag{43}$$

the covariance of the predicted measurement is

$$\mathbf{\Sigma}_{\hat{\mathbf{y}}_j} = \mathbf{J}_{f,j}\,\mathbf{\Sigma}_{\hat{\mathbf{p}}_j}\mathbf{J}_{f,j}{}^\top = \mathbf{J}_{f,j}\mathbf{R}_{t_2,t_1}\mathbf{J}_{f^{-1},i}\,\mathbf{\Sigma}_{\mathbf{z}_i}\,\mathbf{J}_{f^{-1},i}{}^\top\mathbf{R}^\top_{t_2,t_1}\mathbf{J}_{f,j}{}^\top. \tag{44}$$

2.4.3 Residual covariance

Since $\mathbf{y}_j$ and $\hat{\mathbf{y}}_j$ are independent, the residual covariance is the sum of the two contributions. If the confidence of the feature-correspondence $\mathbf{\Sigma}_{ij}^{\text{corr}} \in \mathbb{R}^{2\times 2}$ can also be learnt, it enters as an additional independent term,

$$\mathbf{\Sigma}_{\mathbf{r}_{ij}} = \underbrace{\mathbf{\Sigma}_{uv,j}}_{\text{from learning}} + \mathbf{J}_{f,j}\mathbf{R}_{t_2,t_1}\mathbf{J}_{f^{-1},i}\,\mathbf{\Sigma}_{\mathbf{z}_i}\,\mathbf{J}_{f^{-1},i}{}^{\top}\mathbf{R}_{t_2,t_1}^{\top}\mathbf{J}_{f,j}{}^{\top} + \mathbf{\Sigma}_{ij}^{\text{corr}}. \tag{45}$$

Without correspondence uncertainty, the last term is simply dropped.

Note: $\mathbf{J}_{f,j}$ and $\mathbf{J}_{f^{-1},i}$ cancel only when $\mathbf{R} = \mathbf{I}$ and $\mathbf{t} = \mathbf{0}$, i.e., $\mathbf{J}_{f,j}\mathbf{J}_{f^{-1},i} = [\,\mathbf{I}_2 \;\; \mathbf{0}\,]$, which reduces the second term to $\mathbf{\Sigma}_{uv,i}$. In general, the residual covariance is **not** simply $\mathbf{\Sigma}_{uv,j} + \mathbf{\Sigma}_{uv,i}$.

2.4.4 Cost and Hessian

The cost of the feature pair (i, j) is the squared Mahalanobis distance

$$\mathcal{J}_{I,ij} = \mathbf{r}_{ij}^{\top}\mathbf{\Sigma}_{\mathbf{r}_{ij}}^{-1}\mathbf{r}_{ij}, \tag{46}$$

which leads to the normal equation $\mathbf{H}_{I,ij}\,\delta\boldsymbol{\xi} = \mathbf{b}_{I,ij}$ with

$$\mathbf{H}_{I,ij} = \mathbf{J}_{\xi,ij}^{\top}\mathbf{\Sigma}_{\mathbf{r}_{ij}}^{-1}\mathbf{J}_{\xi,ij}, \qquad \mathbf{J}_{\xi,ij} = \frac{\partial \mathbf{r}_{ij}}{\partial \boldsymbol{\xi}} = -\mathbf{J}_{f,j}\,\frac{\partial \hat{\mathbf{p}}_j}{\partial \boldsymbol{\xi}}, \tag{47}$$

where the Jacobian $\mathbf{J}_{\xi,ij}$ contains the spherical projection Jacobian $\mathbf{J}_{f,j}$.

2.5 Intensity-Image Feature 3D Point-to-Point Residual Formulation

The 2D formulation above uses the range only to back-project the feature in the first frame, while the matched feature in the second frame contributes only its image coordinates. Since the LiDAR measures the range of every pixel, we can also back-project the matched feature j and formulate the residual directly in 3D. In this way, each intensity-image feature becomes a 3D LiDAR point that carries a descriptor, and the descriptor matches define point-to-point correspondences without any nearest-neighbour search.

2.5.1 Point-to-point residual

Using the full measurement $\mathbf{z}_j = [u_j, v_j, \rho_j]^{\top}$ of the matched feature, the point-to-point residual is

$$\tilde{\mathbf{r}}_{ij} = \mathbf{p}_j - \hat{\mathbf{p}}_j = f^{-1}(\mathbf{z}_j) - \hat{\mathbf{p}}_j, \tag{48}$$

where $\hat{\mathbf{p}}_j$ is the transformed point of feature i in Equation (41).

2.5.2 Residual covariance

Both terms of $\tilde{\mathbf{r}}_{ij}$ are back-projected intensity-image features, so their covariances follow from Equation (40). For the observed point in the second frame,

$$\boldsymbol{\Sigma}_{\mathbf{p}_j} = \mathbf{J}_{f^{-1},\boldsymbol{j}}\, \boldsymbol{\Sigma}_{\mathbf{z}_j}\, \mathbf{J}_{f^{-1},\boldsymbol{j}}{}^{\top}, \qquad \boldsymbol{\Sigma}_{\mathbf{z}_j} = \begin{bmatrix} \boldsymbol{\Sigma}_{uv,j} & \mathbf{0} \\ \mathbf{0}^{\top} & \sigma^2_{\rho,j} \end{bmatrix}, \tag{49}$$

and for the predicted point, $\boldsymbol{\Sigma}_{\hat{\mathbf{p}}_j}$ is given in Equation (41). Since $\mathbf{p}_j$ and $\hat{\mathbf{p}}_j$ are independent, the residual covariance is

$$\boldsymbol{\Sigma}_{\tilde{\mathbf{r}}_{ij}} = \mathbf{J}_{f^{-1},\boldsymbol{j}}\, \boldsymbol{\Sigma}_{\mathbf{z}_j}\, \mathbf{J}_{f^{-1},\boldsymbol{j}}{}^{\top} + \mathbf{R}_{t_2,t_1} \mathbf{J}_{f^{-1},\boldsymbol{i}}\, \boldsymbol{\Sigma}_{\mathbf{z}_i}\, \mathbf{J}_{f^{-1},\boldsymbol{i}}{}^{\top} \mathbf{R}_{t_2,t_1}^{\top} + \boldsymbol{\Sigma}_{ij}^{\text{corr}}, \tag{50}$$

where $\boldsymbol{\Sigma}_{ij}^{\text{corr}} \in \mathbb{R}^{3\times 3}$ is the confidence of the feature-correspondence expressed in 3D. Compared with Equation (45), the projection Jacobian $\mathbf{J}_{f,j}$ no longer appears, and the range noise of both features enters the residual covariance.

2.5.3 Cost and Hessian

The cost and Hessian of the feature pair (i, j) are

$$\tilde{\mathcal{J}}_{I,ij} = \tilde{\mathbf{r}}_{ij}^{\top} \boldsymbol{\Sigma}_{\tilde{\mathbf{r}}_{ij}}^{-1} \tilde{\mathbf{r}}_{ij}, \qquad \tilde{\mathbf{H}}_{I,ij} = \tilde{\mathbf{J}}_{\xi,ij}^{\top} \boldsymbol{\Sigma}_{\tilde{\mathbf{r}}_{ij}}^{-1} \tilde{\mathbf{J}}_{\xi,ij}, \qquad \tilde{\mathbf{J}}_{\xi,ij} = \frac{\partial \tilde{\mathbf{r}}_{ij}}{\partial \boldsymbol{\xi}} = -\frac{\partial \hat{\mathbf{p}}_j}{\partial \boldsymbol{\xi}}. \tag{51}$$

The Jacobian is the standard point-transformation Jacobian, as in geometric point-to-point ICP, while the descriptor matching provides the correspondences and $\boldsymbol{\Sigma}_{\tilde{\mathbf{r}}_{ij}}$ provides the anisotropic, per-correspondence weighting.

2.6 Discussion

Both formulations provide a principled way of fusing the LiDAR geometric measurements and the LiDAR intensity-image features. Because every residual is weighted by a covariance propagated from the raw measurement noise ($\boldsymbol{\Sigma}_{uv}$ and σ_ρ^2), the geometric and intensity terms are expressed on a common statistical scale, and we can drop the balancing parameter α in Equation (33) and Equation (34) (i.e., $\alpha = 1$). This is possible because both modalities come from the same sensor, so the intensity image shares the LiDAR range and projection model. The formulation therefore applies to LiDAR and LiDAR intensity-image fusion, but not directly to general LiDAR–camera fusion pipelines.

3 Lidar Image Pixel Correspondences under Motion Distortion

In this section, we derive the ground-truth pixel correspondences between two lidar intensity images. Every pixel of a lidar image is one 3D lidar point, so a pixel of image A can be lifted to 3D, moved into the second scan B, and projected back. The images themselves are never warped or undistorted: a correspondence points to where the surface point really appears in the **raw**, motion-distorted image B. Because the sensor moves during a scan, this requires re-applying the motion distortion of scan B, which leads to a small fixed-point problem.

3.1 Frames and Lidar Image Model

We follow the notation of Section 1. We denote the inertial (map) frame, the robot frame, and the lidar sensor frame as $\mathcal{F}_i$, $\mathcal{F}_r$, and $\mathcal{F}_s$, with the fixed extrinsic $\mathbf{T}_{sr}$. The two scans have stamps t_A and t_B, and we write $\mathcal{F}_A \equiv \mathcal{F}_{st_A}$ and $\mathcal{F}_B \equiv \mathcal{F}_{st_B}$ for the sensor frames at the stamps. Following Barfoot (2024), $\mathbf{T}_{ba}$ maps points from $\mathcal{F}_a$ to $\mathcal{F}_b$, a point expressed in $\mathcal{F}_a$ is written $\mathbf{p}^a$, and transforms act on points in homogeneous coordinates, $\mathbf{p}^b = \mathbf{T}_{ba}\,\mathbf{p}^a$.

The lidar image has $H \times W$ pixels (128×1024). We write a pixel as $\mathbf{y} = [u, v]^\top$, where $u \in [0, W)$ is the column (azimuth) and $v \in [0, H)$ is the row (elevation, one per beam). The sensor fires one *measurement column* at a time. Since the image is destaggered, pixel (u, v) was measured in the measurement column $c(\mathbf{y})$ given by

$$c(\mathbf{y}) = \big(\lfloor u \rceil - s_{\lfloor v \rceil}\big) \bmod W, \tag{52}$$

where $\lfloor \cdot \rceil$ rounds to the nearest integer and s_v is the per-row pixel shift ($s_v \in \{-12, -4, 4, 12\}$ for our sensor, see the `pixel_shift_by_row` in the `meta.json` file). All pixels of one measurement column share a firing time t_c, but the pixels of one image column do not. Within scan A, column c fires at $t_{A,c} \in [t_A, t_A + T_{\text{scan}})$, with $T_{\text{scan}} = 0.1\,\text{s}$. We denote the sensor frame at that time as $\mathcal{F}_{A,c}$, and similarly $t_{B,c}$ and $\mathcal{F}_{B,c}$ for scan B.

The operator $(\cdot) \bmod W$ wraps column indices around the image boundary, reflecting the lidar's $360°$ horizontal field of view. During destaggering, the Ouster SDK (Ouster Inc., 2024) circularly shifts each row v of the raw image by s_v pixels using `np.roll()`:

```
for row in range(H):
    destaggered[row, :] = np.roll(staggered[row, :], pixel_shift_by_row[row])
```

A positive shift moves pixels to the right, while a negative shift moves them to the left. Pixels that move beyond either image boundary wrap around to the opposite side. For example, shifting a row by $+2$ moves the pixel originally at column $c = 0$ to column $u = 2$. Therefore, a pixel acquired at measurement column c appears at column $u = (c + s_v) \bmod W$ in the destaggered image. Eq. (52) simply **reverses this operation**, recovering the original measurement column $c = (u - s_v) \bmod W$ from the destaggered pixel location. This is necessary because the acquisition timestamp is associated with the measurement column c, rather than the destaggered image column u.

3.1.1 Ouster beam model and inverse projection

The Ouster driver computes each 3D point from a measurement $\mathbf{z} = [u, v, \rho]^\top$, i.e., a pixel and its measured range ρ. The calibration (`beam_intrinsics` in `meta.json`) gives each beam k an altitude angle α_k and an azimuth offset β_k, and all beams start from a beam origin that is offset by $n = 15.8\,\text{mm}$ from the sensor axis. Measurement column c is fired at the encoder angle $\epsilon_c = 2\pi(1 - c/W)$, which decreases with c (the sweep is clockwise seen from above). Beam v then points at the azimuth $\theta = \epsilon_c - \beta_v$ and the elevation $\phi = \alpha_v$, and the point is the beam origin plus $(\rho - n)$ along the beam,

$$\mathbf{p} = f^{-1}(\mathbf{z}) = \begin{bmatrix} (\rho - n)\cos\theta\cos\phi + n\cos\epsilon_c \\ (\rho - n)\sin\theta\cos\phi + n\sin\epsilon_c \\ (\rho - n)\sin\phi \end{bmatrix}, \qquad c = c(\mathbf{y}). \tag{53}$$

The encoder angle comes from the **measurement** column c, not from the image column u; using u would rotate the points of our sensor by up to 4.5° ($\approx 13\,\text{px}$). Equation (53) reproduces the driver's points to below $10^{-4}\,\text{deg}$, so $f^{-1}(\mathbf{z}_i)$ in Equation (65) is simply the published point. (The stored range $\|\mathbf{p}\|$ differs from ρ by at most $1.2\,\text{mm}$ because of n.)

Why the destaggered image looks like a panorama. Substituting $c = u - s_v$ into $\theta = \epsilon_c - \beta_v$ gives the azimuth of pixel (u, v),

$$\theta = 2\pi\left(1 - \frac{u}{W}\right) + \delta_v, \qquad \delta_v = \frac{2\pi s_v}{W} - \beta_v. \tag{54}$$

The shifts are chosen as $s_v \approx \beta_v W/2\pi$ (e.g., $\beta_0 = 4.21^\circ$ and $s_0 = 12$ columns $= 4.22^\circ$), so the residual δ_v is only 0.01–0.03° ($< 0.1\,\text{px}$). An image column is therefore (almost) one azimuth for all rows, but its pixels come from different firings. For example, the pixels of image column $u = 100$ in rows 0–3 were measured in four firings:

Row v	β_v [deg]	s_v [col]	δ_v [deg]	$c([100, v]^\top)$
0	4.21	12	0.009	88
1	1.38	4	0.026	96
2	−1.43	−4	0.024	104
3	−4.23	−12	0.011	112

Row 0 points 4.21° ahead of the sweep and reaches the azimuth of column 100 already at firing 88, while row 3 points 4.23° behind and reaches it only at firing 112. The four pixels are measured up to 24 firings ($\approx 2.3\,\text{ms}$) apart, which is why the firing time must come from $c(\mathbf{y})$.

3.1.2 Projection model

The projection $f(\mathbf{p})$ maps an arbitrary 3D point in the lidar frame, not necessarily a measured one, to a continuous (**sub-pixel**) location $\mathbf{y} = [u, v]^\top$ in the destaggered image. Integer coordinates are pixel centres.

The starting point is the spherical projection used by COIN-LIO (Pfreundschuh et al., 2024),

$$\theta = \operatorname{atan2}(y, x), \qquad \phi = \operatorname{atan2}\left(z, \sqrt{x^2 + y^2}\right), \qquad u = W\left(1 - \frac{\theta}{2\pi}\right), \qquad v = \text{row of elevation } \phi. \tag{55}$$

Our $f(\cdot)$ is the exact inverse of Equation (53), which adds three corrections to it:

1. **Measured beam altitudes.** The altitudes $\alpha_0 > \cdots > \alpha_{H-1}$ ($20.9°$ to $-21.8°$) are not evenly spaced (gaps of 0.26–0.37°), so v is interpolated linearly between the two neighbouring beams, $v = k + (\alpha_k - \phi)/(\alpha_k - \alpha_{k+1})$, and extrapolated outside the field of view. (α_k are the calibrated altitude angles that is provided as the `beam_altitude_angles` array in the `meta.json` file)
2. **Per-row azimuth residual.** Solving Equation (54) for u gives $u = [W\left(1 - (\theta - \delta_{\lfloor v \rceil})/2\pi\right)] \bmod W$. Note, $\delta_{\lfloor v \rceil}$ can be directly computed from the json file parameter. $\delta_{\lfloor v \rceil} = (\frac{2\pi s_{\lfloor v \rceil}}{W}) - \beta_{\lfloor v \rceil}$, where $s_{\lfloor v \rceil}$ is the pixel shift `pixel_shift_by_row[v]` in pixels and $\beta_{\lfloor v \rceil}$ is the corresponding azimuth offset, `beam_azimuth_angles[v]` in radians.
3. **Beam origin offset.** θ and ϕ are measured from the beam origin $n\,[\cos\epsilon, \sin\epsilon, 0]^\top$, not from the sensor origin. The encoder angle $\epsilon = \theta + \beta_{\lfloor v \rceil}$ depends on the unknown row and azimuth, so it is found by a fixed-point iteration (Algorithm 1, four iterations starting from $\epsilon = \operatorname{atan2}(y, x)$).

In compact form, with $d_x = x - n\cos\epsilon$ and $d_y = y - n\sin\epsilon$ and iterating on ϵ to convergence,

$$\begin{aligned} &\theta = \operatorname{atan2}(d_y, d_x), \qquad \phi = \operatorname{atan2}\left(z, \sqrt{d_x^2 + d_y^2}\right), \\ &v = \operatorname{interp}\left(\phi; \{\alpha_k\}\right) = k + \frac{\alpha_k - \phi}{\alpha_k - \alpha_{k+1}}, \qquad \epsilon = \theta + \beta_{\lfloor v \rceil}, \\ &\mathbf{y} = f(\mathbf{p}) = \begin{bmatrix} u \\ v \end{bmatrix}, \qquad u = \left[W\left(1 - \frac{\theta - \delta_{\lfloor v \rceil}}{2\pi}\right)\right] \bmod W. \end{aligned} \tag{56}$$

Note, for the k in the interpolation formula $v = k + (\alpha_k - \phi)/(\alpha_k - \alpha_{k+1})$, it is the index of the beam immediately above the elevation ϕ. We need to look through the look up table of `beam_altitude_angles` to find it. By construction, $f\left(f^{-1}(\mathbf{z})\right) = [u, v]^\top$.

Note, the computation of the pixel $y = [u, v]^\top$ is sub-pixel accurate

$$\begin{aligned} v &= k + \frac{\alpha_k - \phi}{\alpha_k - \alpha_{k+1}} \\ u &= \left[W\left(1 - \frac{\theta - \delta_{\lfloor v \rceil}}{2\pi}\right)\right] \bmod W. \end{aligned} \tag{57}$$

3.2 Per-Column Sensor Poses

The lidar odometry estimates a continuous-time trajectory $\{\mathbf{T}_{ri}(t), \boldsymbol{\varpi}(t)\}$ with the white-noise-on-acceleration (constant-velocity) Gaussian-process prior (Anderson et al., 2015; Barfoot, 2024). The state at

Algorithm 1 Lidar projection $f(\mathbf{p})$, Equation (56) (`OusterModel.project`)

function $f(\mathbf{p} = [x, y, z]^\top)$ ▷ $\mathbf{p}$ in the lidar frame at the firing time
  $\epsilon \leftarrow \operatorname{atan2}(y, x)$ ▷ initial encoder angle: ignore the beam origin offset
  for 4 iterations **do**
    $d_x \leftarrow x - n\cos\epsilon, \quad d_y \leftarrow y - n\sin\epsilon$ ▷ point relative to the beam origin
    $\theta \leftarrow \operatorname{atan2}(d_y, d_x)$ ▷ beam azimuth
    $\phi \leftarrow \operatorname{atan2}\left(z, \sqrt{d_x^2 + d_y^2}\right)$ ▷ beam elevation
    $v \leftarrow k + \dfrac{\alpha_k - \phi}{\alpha_k - \alpha_{k+1}}$ with k such that $\alpha_k \geq \phi \geq \alpha_{k+1}$ ▷ fractional row; same formula with $k = 0$ or $k = H - 2$ outside the FOV
    $\bar{v} \leftarrow \operatorname{clip}\left(\lfloor v \rceil, 0, H - 1\right)$ ▷ nearest beam
    $\epsilon \leftarrow \theta + \beta_{\bar{v}}$ ▷ encoder angle at which beam $\bar{v}$ points along θ
  end for
  $u \leftarrow \left\lceil W\left(1 - \dfrac{\theta - \delta_{\bar{v}}}{2\pi}\right)\right\rceil \bmod W$ ▷ Equation (54) solved for u
  return $\mathbf{y} = [u, v]^\top$ ▷ sub-pixel; integers are pixel centres
end function

each knot k is the pose $\mathbf{T}_k = \mathbf{T}_{ri}(t_k)$ and the body velocity $\boldsymbol{\varpi}_k$, following the motion model of Section 1,

$$\dot{\mathbf{T}}_{ri}(t) = \boldsymbol{\varpi}^\wedge \mathbf{T}_{ri}(t). \tag{58}$$

For $t \in [t_k, t_{k+1}]$, with $\tau = t - t_k$ and $\Delta t = t_{k+1} - t_k$, the pose is interpolated as

$$\begin{aligned} \boldsymbol{\xi}_{k+1,k} &= \ln\left(\mathbf{T}_{k+1}\mathbf{T}_k^{-1}\right)^\vee, \qquad \boldsymbol{\gamma}_{k+1} = \mathcal{J}^{-1}(\boldsymbol{\xi}_{k+1,k})\,\boldsymbol{\varpi}_{k+1}, \\ \boldsymbol{\xi}(\tau) &= \Lambda_{12}(\tau)\,\boldsymbol{\varpi}_k + \Omega_{11}(\tau)\,\boldsymbol{\xi}_{k+1,k} + \Omega_{12}(\tau)\,\boldsymbol{\gamma}_{k+1}, \\ \mathbf{T}_{ri}(t) &= \exp\left(\boldsymbol{\xi}(\tau)^\wedge\right)\mathbf{T}_k, \end{aligned} \tag{59}$$

with the 2×2 coefficient matrices

$$\begin{aligned} \boldsymbol{\Omega}(\tau) &= \mathbf{Q}(\tau)\,\boldsymbol{\Phi}(\Delta t - \tau)^\top \mathbf{Q}(\Delta t)^{-1}, \qquad \boldsymbol{\Lambda}(\tau) = \boldsymbol{\Phi}(\tau) - \boldsymbol{\Omega}(\tau)\,\boldsymbol{\Phi}(\Delta t), \\ \mathbf{Q}(\tau) &= \begin{bmatrix} \tau^3/3 & \tau^2/2 \\ \tau^2/2 & \tau \end{bmatrix}, \qquad \boldsymbol{\Phi}(\tau) = \begin{bmatrix} 1 & \tau \\ 0 & 1 \end{bmatrix}. \end{aligned} \tag{60}$$

Outside the knots, the trajectory is extrapolated with the velocity of the nearest knot,

$$\mathbf{T}_{ri}(t) = \exp\left((t - t_k)\boldsymbol{\varpi}_k^\wedge\right)\mathbf{T}_k. \tag{61}$$

Equations (59)–(61) are the interpolation and extrapolation of the STEAM constant-velocity trajectory interface, so $\mathbf{T}_{ri}(t)$ and $\boldsymbol{\varpi}(t)$ are what the STEAM pose and velocity interpolators return. The frame i here is the odometry (map) frame, called m in the code.

- vtr (C++ STEAM, `odometry_icp_module.cpp`): `T_rintp_m_eval`, from

`trajectory->getPoseInterpolator(t)`.

- Python port (`core/odometry_icp.py`): `T_rc_m`, from `sample_trajectory`, which evaluates the same equations for all W column times at once ($\sim$10$\times$ faster than one pysteam interpolator per column). It agrees with pysteam's pose and velocity interpolators to $\sim 10^{-14}$ (checked in `tests/test_trajectory.py`).

3.2.1 Sensor pose and relative sensor motion

We write the time-dependent sensor pose compactly as

$$\mathbf{T}(t) \triangleq \mathbf{T}_{si}(t) = \mathbf{T}_{sr}\,\mathbf{T}_{ri}(t), \tag{62}$$

and the relative sensor transformation between any two timestamps as

$$\mathbf{T}(t_2, t_1) \triangleq \mathbf{T}(t_2)\,\mathbf{T}(t_1)^{-1} = \mathbf{T}_{sr}\,\mathbf{T}_{ri}(t_2)\,\mathbf{T}_{ri}(t_1)^{-1}\,\mathbf{T}_{sr}^{-1}, \qquad \mathbf{p}_s(t_2) = \mathbf{T}(t_2, t_1)\,\mathbf{p}_s(t_1). \tag{63}$$

The first argument is the destination timestamp and the second is the source timestamp. Both are lidar sensor frames.

3.2.2 Per-column motion within a scan

Evaluating Equation (63) at every column firing time gives, for each scan, one transform per measurement column, from the sensor frame at the stamp to the sensor frame at the firing time,

$$\mathbf{T}_c^A \triangleq \mathbf{T}(t_{A,c}, t_A), \qquad \mathbf{T}_c^B \triangleq \mathbf{T}(t_{B,c}, t_B), \qquad c = 0, \ldots, W-1. \tag{64}$$

The superscript names the scan and the subscript the measurement column. The stamp is the firing time of the first column, so $\mathbf{T}_0^A = \mathbf{I}$. Each $\mathbf{T}_c^A$ is evaluated from the interpolated trajectory (Equations (59)–(61)) at the column's own firing time. The odometry stores the $W = 1024$ transforms with each scan (`col_T_sc_s`), and removing or re-applying the motion distortion (Section 3.3) uses them directly.

3.3 Removing and Re-Applying the Motion Distortion

Pixel i of scan A, with $\mathbf{z}_i = [u_i, v_i, \rho_i]^\top$, is measured in the sensor frame at its own firing time, $\mathbf{p}_i^{A,c_i} = f^{-1}(\mathbf{z}_i)$ in $\mathcal{F}_{A,c_i}$, with $c_i = c(\mathbf{y}_i)$. Removing the motion distortion (undistorting) expresses it in the single rigid frame of the scan stamp,

$$\mathbf{p}_i^A = \left(\mathbf{T}_{c_i}^A\right)^{-1} f^{-1}(\mathbf{z}_i). \tag{65}$$

The reverse operation, re-applying the motion distortion (re-distorting), moves a point from the stamp frame into the frame in which column c measured it,

$$\mathbf{p}^{B,c} = \mathbf{T}_c^B\,\mathbf{p}^B. \tag{66}$$

Each saved scan stores the undistorted points $\mathbf{p}_i^A$ for all pixels.

3.4 Relative Pose between the Two Scans

The odometry gives the initial guess of the relative pose between the two stamp frames,

$$\check{\mathbf{T}}_{BA} = \mathbf{T}(t_B, t_A). \tag{67}$$

We refine it with multi-scale point-to-plane ICP between the two undistorted point clouds, which gives $\mathbf{T}_{BA}$. Since both clouds are rigid snapshots at their stamps, a single rigid transform is the correct model. The pair is rejected if the ICP fitness is low (too little overlap), if ICP moves the pose by more than $0.3\,\text{m}$ or 3° from the odometry guess, or if fewer than 2000 pixels get a correspondence.

Note: the per-column motions $\mathbf{T}_c^A$ and $\mathbf{T}_c^B$ come from the GP trajectory, while the relative pose $\mathbf{T}_{BA}$ used below is the **ICP-refined** one, not $\mathbf{T}(t_B, t_A)$ from the trajectory. The odometry drifts over long gaps between the scans, but the motion within one scan (0.1 s) is accurate.

3.5 Predicted Pixel in the Raw Image B

Pixel i of A is first moved into the stamp frame of B,

$$\mathbf{q}_i \triangleq \mathbf{p}_i^B = \mathbf{T}_{BA}\,\mathbf{p}_i^A. \tag{68}$$

Scan B did not see $\mathbf{q}_i$ from $\mathcal{F}_B$, but from $\mathcal{F}_{B,c}$, where c is the measurement column that fired towards the point. The predicted pixel in B is

$$\boxed{\hat{\mathbf{y}}_j = f\left(\mathbf{T}_{c^\star}^B\,\mathbf{T}_{BA}\,\mathbf{p}_i^A\right)} \tag{69}$$

Reading from right to left: apply the ICP-refined relative pose to the undistorted point of A (from $\mathcal{F}_A$ to $\mathcal{F}_B$), re-distort it with the motion of B (to $\mathcal{F}_{B,c^\star}$), and project into the raw image B.

To make the whole chain from the raw pixel of A explicit, substituting Equation (65) gives

$$\hat{\mathbf{y}}_j = f\left(\mathbf{T}_{c^\star}^B\,\mathbf{T}_{BA}\left(\mathbf{T}_{c_i}^A\right)^{-1} f^{-1}(\mathbf{z}_i)\right), \tag{70}$$

i.e., back-project the pixel of A (in $\mathcal{F}_{A,c_i}$) and undistort it with the motion of A (to $\mathcal{F}_A$) before the steps above. **Note:** Equation (70) only shows the relationship; the two right-most steps are never computed when building the pairs. Every pixel of the organized point cloud published by the driver (`/ouster/points`) already comes with its 3D point $\mathbf{p}_i^{A,c_i}$, so f^{-1} is not evaluated at all. The odometry run undistorts these points once, with Equation (65), and stores $\mathbf{p}_i^A$ for every pixel (`xyz_undist`). The pixel correspondences therefore start from the stored pair $(\mathbf{z}_i, \mathbf{p}_i^A)$ and evaluate only Equation (69): the relative pose, the re-distortion with the motion of B, and the projection f.

3.5.1 The circular dependency and the fixed point

Equation (69) cannot be evaluated directly, because the predicted pixel and the measurement column depend on each other. The pixel depends on the column, through the sensor pose at the column's firing time; and

the column depends on the pixel, through Equation (52):

$$\hat{\mathbf{y}}_j = f\left(\mathbf{T}_{c^\star}^B\, \mathbf{q}_i\right), \qquad c^\star = c\left(\hat{\mathbf{y}}_j\right). \tag{71}$$

Both have to hold at the same time. This is the physical condition that the sensor was at $\mathcal{F}_{B,c^\star}$ at the moment its beam swept across the point. Any other column is self-contradictory: re-distorting with $\mathbf{T}_c^B$ assumes the point was hit at $t_{B,c}$, but the pixel it lands on was measured by column $c(\hat{\mathbf{y}}_j) \neq c$, at a different time.

We solve Equation (71) by alternating between the two relations,

$$\hat{\mathbf{y}}^{(k)} = f\left(\mathbf{T}_{c^{(k)}}^B\, \mathbf{q}_i\right), \qquad c^{(k+1)} = c\left(\hat{\mathbf{y}}^{(k)}\right), \qquad c^{(0)} = c\left(f(\mathbf{q}_i)\right), \tag{72}$$

i.e., the initial column is the one $\mathbf{q}_i$ projects to without any motion. Two details matter. First, every pass re-distorts the original $\mathbf{q}_i$, never the point of the previous pass, because $\mathbf{T}_c^B$ only applies to points in $\mathcal{F}_B$. Second, the projected pixel gives an image column $\hat{u}$, which Equation (52) converts to a measurement column (using the row $\hat{v}$) before its firing time can be used.

After the last pass, the final projection gives the sub-pixel match $\hat{\mathbf{y}}_j = f\left(\mathbf{T}_{c^\star}^B\, \mathbf{q}_i\right)$ and the predicted range $\hat{\rho}_j = \left\|\mathbf{T}_{c^\star}^B\, \mathbf{q}_i\right\|$. The transform $\mathbf{T}_c^B$ is constant within a measurement column, so $\hat{\mathbf{y}}_j$ is piecewise continuous in c, with steps of about 10^{-3} px between neighbouring columns ($T_{\text{scan}}/W \approx 98\,\mu$s apart).

Convergence test. A point on a pixel border may alternate between two rows, whose measurement columns differ by up to the shift spread $\max_v s_v - \min_v s_v = 24$ columns (about 2.4 ms). We therefore accept the iteration as converged if

$$\left|c^{(K)} - c^{(K-1)}\right| \leq \max_v s_v - \min_v s_v + 1, \tag{73}$$

where the difference is **not** taken modulo W, so a jump across the seam ($\approx W$ columns, a full scan apart in time) is not mistaken for convergence.

3.5.2 The azimuth seam

The sensor fires the columns in time order, $c = 0 \to W-1$. Column $W-1$ is next to column 0 in azimuth, but they fire almost a full scan apart, so $\mathbf{T}_0^B$ and $\mathbf{T}_{W-1}^B$ differ by one scan of motion. Therefore the column returned by one pass of Equation (72) jumps when $c^{(k)}$ crosses the seam. In a band of width

$$\Delta_{\text{seam}} \approx W\kappa = \frac{W\, T_{\text{scan}}\left(|\omega_z| + v_\perp/\rho\right)}{2\pi} \quad \text{columns} \tag{74}$$

Equation (71) has an unusual number of solutions:

- **Overlap** (the sweep covers more than 360°): two fixed points, $c^\star \approx 0$ and $c^\star \approx W-1$. Scan B saw the surface twice, and both are valid observations.
- **Gap** (the sweep covers less than 360°): no fixed point. Scan B never measured this direction, and the iteration alternates between $c \approx 0$ and $c \approx W-1$.

Whether the band is an overlap or a gap depends on whether the platform rotates with or against the spin direction, and on the translation for close points. To handle both cases, points with $c^{(0)} < m$ or $c^{(0)} \geq W - m$ (seam margin $m = 50$ columns) are also iterated from the other end of the scan, $c^{(0)} = W - 1$ or $c^{(0)} = 0$. Of the two candidates, we keep the one with the smaller range residual (Equation (75)). Points in the gap fail the convergence test and are dropped. On real pairs, the matches dropped in this way were inconsistent by 4.7 px (median) when re-distorted with the firing time of the pixel they land on.

Algorithm 2 Pixel correspondences $A \to B$

for each valid pixel i of A with $\rho_i > \rho_{\min}$ **do**
  $\mathbf{q}_i \leftarrow \mathbf{T}_{BA}\, \mathbf{p}_i^A$
  $c^{(0)} \leftarrow c(f(\mathbf{q}_i))$
  for $k = 0, \ldots, K-1$ **do**
    $c^{(k+1)} \leftarrow c\left(f(\mathbf{T}^B_{c^{(k)}}\, \mathbf{q}_i)\right)$ ▷ re-distort $\mathbf{q}_i$, never the previous point
  end for
  $\hat{\mathbf{y}}_j \leftarrow f(\mathbf{T}^B_{c^{(K)}}\, \mathbf{q}_i), \quad \hat{\rho}_j \leftarrow \|\mathbf{T}^B_{c^{(K)}}\, \mathbf{q}_i\|$
  if $c^{(0)}$ is within m columns of the seam **then**
    repeat from the other end of the scan and keep the candidate with the smaller range residual
  end if
  keep $\hat{\mathbf{y}}_j$ if converged, inside the FOV, and it passes the range check
end for
compute $B \to A$ in the same way, and keep the matches that pass the cycle check

3.6 Validity Checks

A predicted pixel $\hat{\mathbf{y}}_j$ is kept as a correspondence only if all of the following hold.

1. **Field of view and return:** $\hat{v}_j \in (-0.5, H - 0.5)$, and B has a return at the nearest pixel $\lfloor \hat{\mathbf{y}}_j \rceil$.
2. **Occlusion check:** the range measured by B at that pixel agrees with the predicted range,

$$|\rho_B\left(\lfloor \hat{\mathbf{y}}_j \rceil\right) - \hat{\rho}_j| < \epsilon_{\text{abs}} + \epsilon_{\text{rel}}\, \hat{\rho}_j, \qquad \epsilon_{\text{abs}} = 0.1\,\text{m}, \quad \epsilon_{\text{rel}} = 0.02. \tag{75}$$

   If B measured a closer surface at that pixel, the point is occluded in B.
3. **Minimum range:** $\rho_i > \rho_{\min} = 1\,\text{m}$, which removes the robot body and ambiguous projections.
4. **Cycle consistency:** the $B \to A$ correspondence of the nearest pixel $\lfloor \hat{\mathbf{y}}_j \rceil$ returns within 1 px of pixel i, with the column difference wrapped to $[-W/2, W/2)$.

3.7 Interpretation of the Sub-Pixel Correspondences

Pixel i of A is always an integer pixel, while its match $\hat{\mathbf{y}}_j$ in B is sub-pixel. $\hat{\mathbf{y}}_j$ is where image B would have recorded the surface point, between the beam directions that B actually sampled. The intensity of B at the match is therefore an interpolation, e.g., bilinear between the four neighbouring pixels, which is accurate on smooth surfaces and less so at depth discontinuities.

Note: the correspondences are stored as $(\text{row}, \text{col}) = (\hat{v}_j, \hat{u}_j)$, i.e., in the reverse order of $\mathbf{y} = [u, v]^\top$. The column is periodic: $\hat{u}_j \in [0, W)$, and $\hat{u}_j \in (W - 1, W)$ lies between the last and the first column, so

interpolation must wrap in azimuth. For the upside-down mount, the stored images and correspondences are rotated by $180°$, $(\hat{v}_j, \hat{u}_j) \mapsto (H - 1 - \hat{v}_j,\ (W - 1 - \hat{u}_j) \bmod W)$.

3.8 Validation

On simulated scans with known motion (1.1–1.9 m/s, up to 0.7 rad/s, 1 cm range noise), the correspondences computed from the estimated trajectory and relative pose agree with those computed from the true motion to 0.13–0.18 px (median) beyond 3 m, and 0.68 px between 1 and 3 m. Without motion compensation, the median error is 1.2 px. Pairing a scan with itself ($\mathbf{T}_{BA} = \mathbf{I}$) maps every pixel onto itself with less than 10^{-4} px error.

Table 1: Symbols and their names in the implementation (`make_pairs.py`, `core/motion_distortion.py`, `core/ouster_model.py`).

Symbol	Implementation
$\mathbf{p}_i^A$	`xyz_undist`
$\mathbf{T}(t)$	`T_s_m`
$\mathbf{T}_c^A$, $\mathbf{T}_c^B$	`col_T_sc_s[c]`
$\check{\mathbf{T}}_{BA}^{-1}$, $\mathbf{T}_{BA}^{-1}$	`T_AB_odom`, `T_AB`
$c(\mathbf{y})$	`OusterModel.meas_col`
$f(\cdot)$	`OusterModel.project`
Equation (65), re-distortion	`mdist`, `re_mdist`
Equation (72)	`predict_in_dst`
K, m, $\rho_{\min}$	`re_mdist_iters`, `seam_margin`, `corr_min_range`
$\hat{\mathbf{y}}_j$ (row, col)	`corr_AB`

References

Anderson S and Barfoot TD (2013) Ransac for motion-distorted 3d visual sensors. In: *2013 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, pp. 2093–2099.

Anderson S, MacTavish K and Barfoot TD (2015) Relative continuous-time slam. *The International Journal of Robotics Research* 34(12): 1453–1479.

Barfoot TD (2024) *State estimation for robotics: Second edition*. Cambridge University Press.

Ouster Inc (2024) Ouster SDK documentation: Staggering and destaggering. https://static.ouster.dev/sdk-docs/reference/lidar-scan.html#staggering-and-destaggering. Accessed: 2026-10-26.

Pfreundschuh P, Oleynikova H, Cadena C, Siegwart R and Andersson O (2024) COIN-LIO: Complementary intensity-augmented LiDAR inertial odometry. In: *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, pp. 1730–1737.

Tong CH, Anderson S, Dong H and D Barfoot T (2014) Pose interpolation for laser-based visual odometry. *Journal of Field Robotics* 31(5): 731–757.